



NEKO//DEX

DIRECTORIO FELINO

Manual técnico

Versión 1.0 · septiembre de 2026 · código en `main` · Flutter 3.41.9

Prueba técnica Mobile · Nextep Innovation
Luis Isturiz

Contenido

1. Resumen del sistema

2. Stack y dependencias

3. Entorno de desarrollo

4. Ambientes: DEV, QA y PROD

5. Arquitectura

6. Gestión de estado

7. Navegación y deep links

8. Datos y red

9. Caché y offline-first

10. Notificaciones locales

11. Interfaz y diseño

12. Internacionalización y accesibilidad

13. Rendimiento

14. Pruebas

15. Integración continua y entrega

16. Mantenimiento

17. Problemas conocidos y soluciones

18. Decisiones de diseño y límites

Este manual describe cómo está construida NekoDex (`cat_directory_app`), cómo se levanta el entorno de desarrollo y cómo se mantiene. Está dirigido a quien vaya a revisar, compilar o extender el proyecto. La guía de uso de la app está en el [manual de usuario](#).

1. Resumen del sistema

Qué es	Directorio de razas de gatos sobre la API pública de Catfact Ninja , con ficha por raza y dato curioso aleatorio
Plataformas	Android 7.0+ (API 24) e iOS 13+
Framework	Flutter 3.41.9 · Dart 3.11.5, fijados con FVM (<code>.fvmrc</code>)
Paquete Dart	<code>cat_directory_app</code> (el nombre <code>cat-directory-app</code> de la prueba, con guiones bajos porque Dart no admite guiones)
Ids de la app	Android <code>com.luisturiz.cat_directory_app</code> · iOS <code>com.luisturiz.catDirectoryApp</code> (+ <code>.dev</code> / <code>.qa</code> en los otros ambientes)
Idiomas	Español (base) e inglés
Repositorio	github.com/aliatore/cat-test · binario en Releases

Funcionalidad principal: listado con scroll infinito y control de concurrencia, pull to refresh, búsqueda local con debounce, ruta `/breed/:name` con GoRouter, offline-first con stale-while-revalidate, detalle con dato curioso de carga independiente, reintentos con backoff exponencial, caché con invalidación explícita, deep links verificados, modo oscuro, Hero, notificación local diaria y revalidación al volver a primer plano.

2. Stack y dependencias

Área	Paquete	Uso
Estado	<code>bloc</code> 9, <code>flutter_bloc</code> 9	BLoC y Cubit
Estado	<code>bloc_concurrency</code> 0.3, <code>stream_transform</code> 2	<code>droppable</code> , <code>restartable</code> y <code>debounce</code> de eventos
Navegación	<code>go_router</code> 17	Rutas declarativas, ruta parametrizada y deep links
Red	<code>dio</code> 5	Cliente HTTP con interceptores
Persistencia	<code>hive_ce</code> 2	Caché clave-valor (fork mantenido de Hive)
Conectividad	<code>connectivity_plus</code> 7	Estado de la red
Modelos	<code>freezed</code> 3, <code>json_serializable</code> 6	Entidades inmutables, uniones selladas y JSON
Inyección	<code>get_it</code> 9	Contenedor de dependencias en la raíz de composición
Experiencia	<code>lottie</code> 3, <code>flutter_animate</code> 4, <code>audioplayers</code> 6	Splash animado, micro-animaciones y efectos de sonido
Notificaciones	<code>flutter_local_notifications</code> 22, <code>timezone</code> , <code>flutter_timezone</code>	Raza del día programada en hora local
Calidad	<code>very_good_analysis</code> 10, <code>bloc_test</code> , <code>mocktail</code>	Reglas de análisis y pruebas
Assets	<code>flutter_launcher_icons</code> , <code>flutter_native_splash</code>	Ícono y splash nativo (generados, ver sección 16)

Las versiones exactas están en `pubspec.yaml` y `pubspec.lock`.

3. Entorno de desarrollo

Requisitos

- **FVM** 3.x [o Flutter 3.41.x en el `PATH`].
- **Android:** Android SDK 36 y JDK 17.
- **iOS:** Xcode 26 y CocoaPods 1.16. `connectivity_plus` 7 usa APIs del SDK de iOS 26, así que con Xcode 16 no compila.
- **Editor:** VS Code con las extensiones recomendadas en `.vscode/extensions.json` [Flutter, Bloc y Markdown Mermaid].

Primeros pasos

```
git clone https://github.com/aliatore/cat-test.git
cd cat-test
fvm install
fvm flutter pub get
fvm flutter run --flavor dev
```

El código generado (freezed, json_serializable y traducciones) está versionado: no hace falta correr `build_runner` para compilar.

VS Code

La carpeta `.vscode/` trae todo configurado y apunta al SDK de FVM (`.fvm/versions/3.41.9`).

Depuración (`launch.json`)

Configuración	Qué hace
NekoDex DEV / QA / PROD	<code>flutter run --flavor <ambiente></code> en modo debug
... [profile mode]	Modo profile, para medir rendimiento
... [release mode]	Modo release, como la app publicada
Tests	Corre toda la suite de <code>test/</code> con el depurador
Adjuntar a la app abierta	<code>flutter attach</code> : para depurar un arranque en frío desde un deep link o una notificación

Sobre `main()` aparecen los accesos *Run / Debug* (DEV), *Debug QA* y *Debug PROD*. En iOS, profile y release solo corren en un iPhone físico.

Tareas (`tasks.json`, *Terminal → Run Task*)

Grupo	Tareas
Setup	Dependencias (<code>pub get</code>)
Codegen	<code>build_runner</code> una vez o en modo <i>watch</i>
L10n	Regenerar traducciones (<code>gen-l10n</code>)
Check	Formato, análisis, pruebas y todo como la CI (en secuencia)
Test	Cobertura (<code>coverage/lcov.info</code>)
Build	APK, App Bundle, iOS sin firma e IPA; cada una pregunta el ambiente
Perf	Traza de scroll en modo profile y análisis de tamaño del APK
Deep link	Abrir una ficha en Android o en el simulador eligiendo qué app la recibe
Clean	<code>flutter clean</code> + <code>pub get</code>

Ajustes (`settings.json`): formato y orden de imports al guardar, regla en 80 columnas y los `.freezed.dart` / `.g.dart` anidados bajo su archivo fuente.

Comandos frecuentes

Para	Comando
Correr en un ambiente	<code>fvm flutter run --flavor dev</code>
Pruebas	<code>fvm flutter test</code>
Cobertura	<code>fvm flutter test --coverage</code>
Análisis	<code>fvm flutter analyze</code>
Formato	<code>fvm dart format lib test integration_test test_driver</code>
Regenerar modelos	<code>fvm dart run build_runner build --delete-conflicting-outputs</code>
Regenerar traducciones	<code>fvm flutter gen-l10n</code>
APK de producción	<code>fvm flutter build apk --release --flavor prod</code>
iOS sin firma	<code>fvm flutter build ios --release --no-codesign --flavor prod</code>

4. Ambientes: DEV, QA y PROD

Cada ambiente es un **flavor nativo**: `productFlavors` en Android y un scheme de Xcode por ambiente en iOS. Así las tres apps se instalan lado a lado. En Dart, el flavor llega por `appFlavor` y se traduce a `AppEnvironment`.

	DEV	QA	PROD
Nombre	NekoDex DEV	NekoDex QA	NekoDex
Android	<code>com.luisturiz.cat_directory_app.dev</code>	<code>...qa</code>	<code>com.luisturiz.cat_directory_app</code>
iOS	<code>com.luisturiz.catDirectoryApp.dev</code>	<code>...qa</code>	<code>com.luisturiz.catDirectoryApp</code>
<code>versionName</code> (Android)	<code>1.0.0-dev</code>	<code>1.0.0-qa</code>	<code>1.0.0</code>
Esquema de deep links	<code>nekodex-dev://</code>	<code>nekodex-qa://</code>	<code>nekodex://</code>
API	<code>https://catfact.ninja</code>	<code>https://catfact.ninja</code>	<code>https://catfact.ninja</code>
Revalidar al volver después de	30 s	1 min	5 min
Logs de red en consola (debug)	sí	sí	no
Cinta en la esquina	DEV (cian)	QA (amarilla)	—

- Sin `--flavor` **se compila prod**, por `default-flavor: prod` en `pubspec.yaml`. Por eso `flutter build apk` y la CI siguen generando la app de producción.
- Catfact Ninja no tiene staging: los tres ambientes apuntan a la misma API, pero la URL queda por ambiente para cuando exista uno.

Dónde vive cada valor

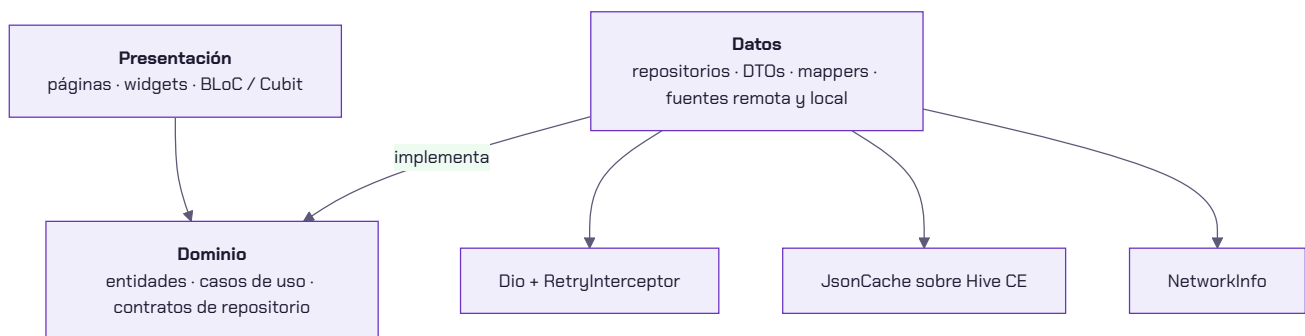
Valor	Archivo
Id, nombre y esquema en Android	<code>android/app/build.gradle.kts</code> [<code>productFlavors</code>]
Id, nombre y esquema en iOS	<code>ios/Flutter/flavors/<ambiente>.xcconfig</code> , incluido desde <code>ios/Flutter/<Modo>-<ambiente>.xcconfig</code>
Configuraciones y schemes de Xcode	<code>Debug-<ambiente></code> , <code>Profile-<ambiente></code> , <code>Release-<ambiente></code> en <code>Runner.xcodeproj</code> ; schemes <code>dev</code> , <code>qa</code> y <code>prod</code>
Lo que usa Dart	<code>lib/core/config/app_environment.dart</code>

Agregar un ambiente (por ejemplo `staging`):

1. Un `create("staging")` en `productFlavors` con su sufijo, nombre y esquema.
2. En iOS: `ios/Flutter/flavors/staging.xcconfig`, los tres `<Modo>-staging.xcconfig`, las tres configuraciones de build en el proyecto (se pueden duplicar desde Xcode), el scheme `staging` y las líneas del `Podfile`; después, `pod install`.
3. El valor `staging` en el enum `AppEnvironment`.
4. Una configuración en `launch.json`. La prueba `test/app/flavors_test.dart` falla si el ambiente existe en Dart pero falta en Android o en iOS.

5. Arquitectura

Clean Architecture organizada por feature. Cada feature tiene sus capas y la regla de dependencias apunta siempre hacia el dominio:



- **Dominio:** entidades [`Breed`, `BreedPage`, `CatFact`], contratos de repositorio y casos de uso. No conoce Flutter, Dio ni Hive.
- **Datos:** DTOs con `json_serializable`, mappers que limpian la respuesta de la API, fuentes remota (Dio) y local (caché) e implementaciones de los repositorios.
- **Presentación:** blocs y cubits, páginas y widgets. Las páginas no conocen el contenedor de dependencias: el router les pasa los blocs ya construidos.

Estructura de carpetas

```
lib/
├─ main.dart                # solo llama a bootstrap()
├─ app/
│   ├─ bootstrap.dart       # arranque: errores, ambiente, dependencias, notificaciones
│   ├─ app.dart             # MaterialApp.router, tema, ciclo de vida, blocs globales
│   ├─ di/injection.dart    # raíz de composición (get_it)
│   ├─ router/app_router.dart # GoRouter: /, /breed/:name y 404
│   └─ view/                # splash animado, cinta de ambiente, ruta desconocida
├─ core/
│   ├─ cache/               # CachePolicy, JsonCache, KeyValueStore
│   ├─ config/              # AppEnvironment, AppConfig
│   ├─ error/               # Result, Failure y su mapeo desde excepciones
│   ├─ network/             # Dio, RetryInterceptor, backoff, NetworkInfo
│   ├─ presentation/        # ConnectivityCubit, textos de error, tiempo relativo
│   ├─ services/            # sonidos y notificaciones (interfaz + implementación)
│   └─ utils/               # slug, log, reloj inyectable
├─ design_system/          # tema claro/oscuro, colores, tipografía y componentes neón
├─ features/
│   ├─ breeds/              # directorio y ficha (domain / data / presentation)
│   ├─ facts/               # dato curioso (domain / data / presentation)
│   ├─ daily_breed/         # programación de la raza del día
│   └─ settings/            # ajustes persistidos (domain / data / presentation)
└─ l10n/                   # ARB en español e inglés y código generado
```

Arranque

`bootstrap()` [`lib/app/bootstrap.dart`] sigue este orden:

1. `WidgetsFlutterBinding.ensureInitialized()` y un manejador global de errores [`PlatformDispatcher.onError`].
2. En debug, un `BlocObserver` que registra cada evento.
3. Resuelve el ambiente [`AppEnvironment.current`] y llama a `configureDependencies`: abre Hive en *Application Support*, crea Dio con la URL del ambiente y registra todas las dependencias.
4. Precarga los sonidos e inicializa las notificaciones [canal Android con los textos localizados].
5. Si la app se abrió tocando una notificación, arranca directamente en esa ficha.
6. `runApp(NekoDexApp(...))`. La carga del directorio empieza mientras corre el splash.

Inyección de dependencias

`lib/app/di/injection.dart` es el único lugar que conoce las implementaciones concretas. Registra como *singleton* la infraestructura [Dio, `NetworkInfo`, `JsonCache`, sonidos, notificaciones], como *lazy singleton* fuentes y repositorios, y como *factory* casos de uso y blocs. `BreedDetailCubit` se registra con parámetros [`registerFactoryParam`] para recibir el slug y la raza inicial.

Modelo de errores

Los errores no viajan como excepciones:

- Los repositorios devuelven `Result<T>`, una clase sellada con `Ok` y `Err`.
- Los fallos son una unión sellada `Failure` [freezed]: `connection`, `timeout`, `server`, `rateLimited` [con `retryAfter`], `request`, `parsing`, `cache`, `notFound` y `unexpected`.
- `mapErrorToFailure` traduce `DioException` y errores de parseo a un `Failure`, y `failure_copy.dart` elige el título, el texto y la ilustración de cada uno.

6. Gestión de estado

Se usa **BLoC** para el directorio y **Cubit** para los estados simples. Por qué BLoC:

- La prueba pide transformar eventos para evitar llamadas duplicadas o fuera de orden. En BLoC eso es parte del modelo: cada evento declara su concurrencia y se prueba como cualquier otro comportamiento.
- Los eventos explícitos dejan rastro: en debug se registra cada evento, y con `bloc_test` cada escenario queda en una prueba legible.
- Para estados sin flujo de eventos (ficha, ajustes, caché) basta un `Cubit`.

BreedsBloc

Evento	Transformador	Qué hace
<code>BreedsStarted</code>	<code>droppable</code>	Emite la página 1 desde la caché y la revalida (stale-while-revalidate)
<code>BreedsNextPageRequested</code>	<code>droppable</code>	Siguiente página del scroll infinito; mientras hay una en curso, las demás se descartan
<code>BreedsRefreshRequested</code>	<code>droppable</code>	Pull to refresh: página 1 desde la red, ignorando la frescura
<code>BreedsRetryRequested</code>	<code>droppable</code>	Reintenta lo que falló: carga inicial, página intermedia o revalidación
<code>BreedsRevalidateRequested</code>	<code>droppable</code>	Revalidación al volver a primer plano
<code>BreedsQueryChanged</code>	<code>debounce(300 ms) + restartable</code>	Búsqueda local; solo cuenta la última
<code>_ConnectivityChanged</code>	—	Al volver la red, reintenta lo pendiente

Detalles del estado:

- `pages` guarda las páginas por número. Un refresh reemplaza la página 1 sin perder las demás, y la lista se arma sin duplicados.
- `failure` / `paginationFailure`: un fallo en la carga inicial bloquea la pantalla, pero uno en una página intermedia solo se muestra en el pie. La lista cargada nunca se borra.
- **Época** (`_epoch`): cada refresh incrementa un contador. Si llega tarde una página pedida antes del refresh, se descarta, para no mezclar datos viejos y nuevos.
- `notice`: avisos de una sola vez (snackbars): datos guardados, actualización fallida, directorio actualizado y conexión restablecida.
- `origin` (`remote` o `cache`), `isStale` y `updatedAt` alimentan el chip de estado y el banner de red.

Otros blocs y cubits

Clase	Responsabilidad
<code>BreedDetailCubit</code>	Resuelve la raza de la ficha: la recibida por la navegación, la memoria, la caché o, en último caso, la API
<code>CatFactBloc</code>	Dato curioso con su propio loading y error; <code>droppable</code> evita pedidos repetidos
<code>ConnectivityCubit</code>	Estado de la red y reintentos en curso ("Reconectando... intento 2 de 3")
<code>BreedsCacheCubit</code>	Resumen de la caché y su borrado desde Ajustes
<code>SettingsCubit</code>	Tema, sonido, raza del día y overlay de rendimiento, persistidos en Hive

`BreedsBloc`, `ConnectivityCubit` y `SettingsCubit` viven por encima del router (`NekoDexApp`), así su estado sobrevive a la navegación. Los de la ficha se crean por ruta.

7. Navegación y deep links

Ruta	Pantalla	Detalle
/	Directorio	Ruta inicial
/breed/:name	Ficha	Sub-ruta de /: al entrar por un deep link la pila queda [directorío, ficha] y <i>atrás</i> vuelve al listado
cualquier otra	Ruta desconocida	<code>errorBuilder</code> con acceso al inicio

- **Normalización:** un `redirect` convierte el parámetro en slug (`/breed/American Curl` → `/breed/american-curl`), así la URL compartida es siempre la misma.
- `extra`: la navegación interna pasa la raza como `extra` para que el Hero tenga destino en el primer frame. La ruta funciona igual sin él, que es el caso de los deep links.
- `ValueKey(slug)` en los providers de la ficha: al saltar de una ficha a otra (deep link o notificación) se crean blocs nuevos y no se ve la raza anterior.

Deep links

Tipo	Formato	Configuración
Esquema propio	<code>nekodex://open/breed/<raza></code> (<code>nekodex-dev://</code> y <code>nekodex-qa://</code> en los otros ambientes)	Intent filter en <code>AndroidManifest.xml</code> y <code>CFBundleURLTypes</code> en <code>Info.plist</code> , con el esquema del flavor
App Links / Universal Links	<code>https://luisturiz.com/breed/<raza></code>	Intent filter con <code>autoVerify</code> e <i>Associated Domains</i> (<code>applinks:\$(DEEP_LINK_HOST)</code> en <code>Runner.entitlements</code>)

Los archivos de verificación están publicados en `https://luisturiz.com/.well-known/assetlinks.json` y `https://luisturiz.com/.well-known/apple-app-site-association` [fuente en `deeplinks/.well-known/`; se sirven desde el repositorio del portafolio]. **Android marca el dominio como `verified` y el link abre la ficha directo en Android e iOS, también con la app cerrada.**

- **Solo PROD está en los archivos:** DEV y QA declaran el mismo dominio pero no verifican, así que se prueban con su esquema propio.
- **Firma:** la huella de `assetlinks.json` es la del certificado del APK publicado en Releases. El APK que genera la CI se firma con la clave de debug del runner y no verifica. Si se firma con otra clave, hay que agregar su huella al archivo publicado.
- **Dominio:** se cambia con `-PdeepLinkHost=...` y `--dart-define=DEEP_LINK_HOST=...`, más `DEEP_LINK_HOST` en los build settings de iOS. Detalle en `deeplinks/README.md`.

```
adb shell pm get-app-links com.luisturiz.cat_directory_app           # estado de la verificación
adb shell am start -a android.intent.action.VIEW -d "https://luisturiz.com/breed/bengal"
xcrun simctl openurl booted "https://luisturiz.com/breed/bengal"
```

8. Datos y red

API consumida

Endpoint	Parámetros	Uso
GET /breeds	<code>page</code> , <code>limit=15</code>	Directorio paginado; la respuesta trae <code>data</code> , <code>current_page</code> , <code>last_page</code> y <code>total</code>
GET /fact	<code>max_length=240</code>	Dato curioso aleatorio; el límite evita textos que rompen la tarjeta

- **Página de 15:** la API tiene 98 razas; con 15 por página el scroll infinito se ejercita de verdad (7 páginas).
- **Limpieza de datos:** el mapper quita las notas al pie (`Foldex[4]` → `Foldex`), separa los paréntesis pegados al nombre, normaliza los espacios y convierte en `null` los campos vacíos, que la UI muestra como "Sin datos". Las razas sin nombre se

descartan.

- **Peticiones compartidas:** si ya hay una petición en vuelo para la misma página, la fuente remota devuelve ese mismo `Future` en lugar de pedirla dos veces.

Ciente HTTP

`buildDio` (`lib/core/network/dio_factory.dart`) crea el cliente con:

- la URL base del ambiente,
- timeouts de 8 s para conectar y 10 s para recibir,
- el `RetryInterceptor`,
- en DEV y QA (solo en debug), un log compacto de una línea por petición.

Política de reintentos (`RetryInterceptor`)

Parámetro	Valor
Reintentos	hasta 3
Espera	exponencial desde 400 ms (400 → 800 → 1600 ms), tope de 4 s
Jitter	50 %: la mitad de la espera es fija y la otra mitad aleatoria, para que los clientes no reintenten a la vez
Se reintentan	errores de conexión, timeouts y respuestas 408, 429, 500, 502, 503 y 504
Métodos	solo idempotentes (<code>GET</code> , <code>HEAD</code> , <code>OPTIONS</code>)
<code>Retry-After</code>	se respeta si no supera 10 s

Cada reintento se publica en `RetryEvents`. `ConnectivityCubit` lo escucha para mostrar el banner "Reconectando...", que desaparece solo.

Conectividad

`ConnectivityNetworkInfo` envuelve `connectivity_plus`. Si no hay ninguna interfaz de red, el repositorio no intenta la petición, así no se gastan reintentos inútiles. En iOS el primer valor puede llegar tarde, por eso espera hasta 600 ms antes de concluir que no hay red. Los cambios de estado se filtran con `debounce` para no reaccionar a parpadeos de la señal.

9. Caché y offline-first

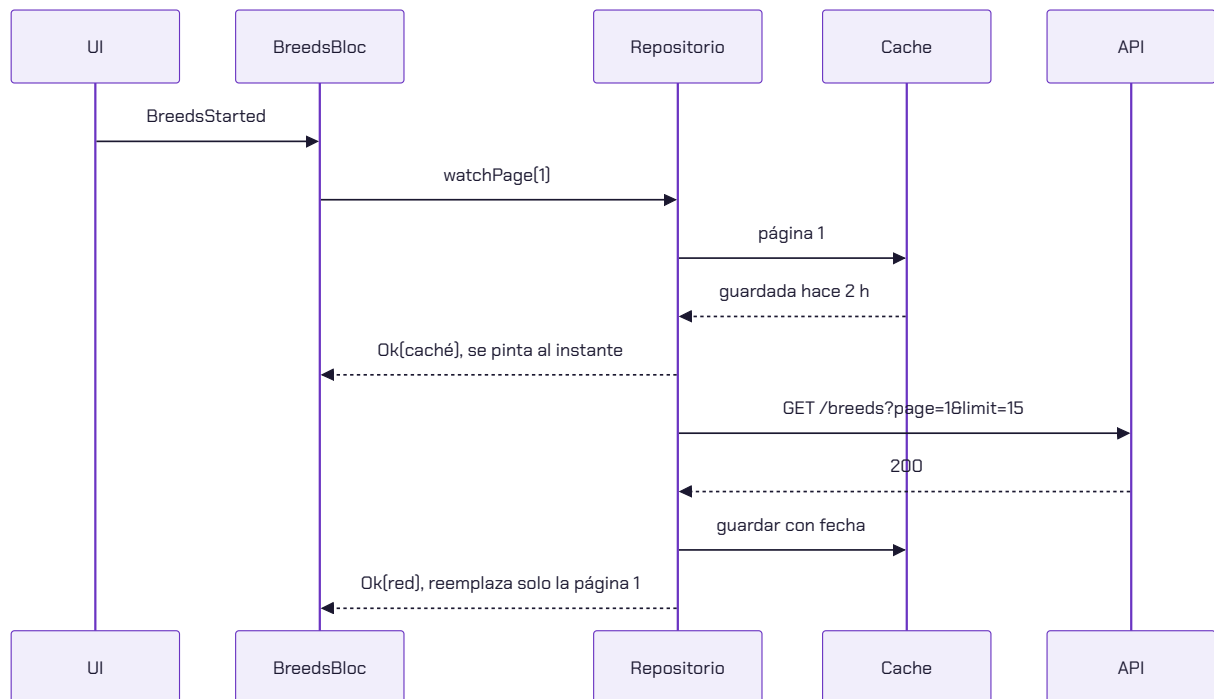
La API responde `Cache-Control: no-cache` y no expone `ETag` ni `Last-Modified`, así que la frescura se decide en el cliente con una política explícita.

Pieza	Detalle
Almacenamiento	Hive CE en <i>Application Support</i> (es caché, no un archivo del usuario): caja <code>cache</code> para datos y <code>settings</code> para ajustes
Formato	<code>JsonCache</code> guarda un sobre <code>{v, t, d}</code> : versión de esquema, fecha de escritura y el JSON
Claves	<code>breeds:<tamaño de página>:<página></code> ; el tamaño va en la clave para no mezclar geometrías entre versiones
Datos curiosos	los últimos 30 vistos, para mostrar uno sin conexión
Versión de esquema	<code>cacheSchemaVersion</code> en <code>injection.dart</code> ; subirla invalida toda la caché tras una actualización

Política (`CachePolicy.breeds`)

Edad de la página	Comportamiento
Menos de 5 min	Fresca: se sirve sin tocar la red

Edad de la página	Comportamiento
De 5 min a 7 días	Vieja: se muestra al instante y se revalida en segundo plano
Más de 7 días	Vencida: se descarta. Al arrancar se purgan todas las vencidas
Pull to refresh	Ignora la frescura y va a la red



Métodos del repositorio (BreedRepository)

- `watchPage` : stream stale-while-revalidate. Emite la caché si no venció; si no estaba fresca, emite después el resultado de la red. Si la red falla tras emitir la caché, el segundo evento es un `Err` y el bloc decide si es bloqueante o solo un aviso.
- `getPage` : para el scroll infinito. Usa la caché fresca, si no la red y, si la red falla, la caché aunque esté vieja.
- `findBySlug` : para deep links y notificaciones. Busca en el índice en memoria, después en la caché y, en último caso, recorre la API página a página, calentando la caché de paso.
- `purgeExpired` , `clearCache` y `cacheInfo` : mantenimiento y la sección de caché de Ajustes.

Revalidación en primer plano: `NekoDexApp` escucha el ciclo de vida con `AppLifecycleListener`. Si la app estuvo en segundo plano más de `revalidateAfter` (5 min en prod), pide `BreedsRevalidateRequested`.

10. Notificaciones locales

Aspecto	Detalle
Librería	<code>flutter_local_notifications</code> 22 con <code>timezone</code> (base de 10 años) y <code>flutter_timezone</code>
Canal Android	<code>daily_breed</code> , con ícono monocromo propio (<code>ic_stat_nekodex</code>) y sonido propio (<code>nekodex_notify</code>)
iOS	Se muestran también con la app abierta; el sonido va en el bundle de Runner
Permiso	Se pide al activar la raza del día o al enviar la de prueba, nunca al abrir la app
Programación	Los próximos 7 días a las 10:00 hora local, ids 1000–1007; la de prueba usa el id 1999
Elección de la raza	Determinista por fecha: la misma fecha da siempre la misma raza
Modo Android	<code>inexactAllowWhileIdle</code> : no exige el permiso de alarmas exactas de Android 14

Aspecto	Detalle
Reprogramación	Con la primera lista buena de cada sesión y, en Android, después de reiniciar el teléfono [<code>ScheduledNotificationBootReceiver</code>]
Toque	Abre <code>/breed/<raza></code> : con la app abierta por <code>onOpenRoute</code> y en frío por <code>launchRoute</code>

Verificación: el flujo completo (permiso, entrega, toque y arranque en frío) está probado en Android. En iOS está configurado y compila, pero falta probarlo en un dispositivo.

11. Interfaz y diseño

- **Tema:** `NekoColors` es un `ThemeExtension` con dos paletas, "Night City" (oscura, la de diseño) y "Distrito de día" (clara, con acentos oscurecidos para pasar contraste AA).
- **Tipografía:** Orbitron para la marca y los títulos, Chakra Petch para el texto y Share Tech Mono para etiquetas y datos. Las tres tienen licencia OFL y están en `assets/fonts/`.
- **Componentes** (`lib/design_system/widgets/`):
 - `NeonPanel`, `NeonButton` (sonido, háptica y semántica de botón), `GlitchText` y `StatusChip`,
 - `StateView` para estados vacíos y de error con ilustración Lottie,
 - `NeonShimmer` para los skeletons,
 - `CatAvatar` : avatar generado por código a partir del patrón de pelaje de la raza, con semilla estable por nombre.
- **Movimiento:** duraciones y curvas centralizadas en `motion.dart`. Con "Reducir movimiento" del sistema se apagan las animaciones decorativas.
- **Sonido:** `audioplayers` con un `AudioPool` por efecto. En iOS usa la categoría *ambient* (respeta el modo silencio y no corta la música); en Android no pide foco de audio.
- **Assets propios:** las animaciones Lottie, los sonidos y el ícono se generan con scripts en `tool/`, sin licencias de terceros (ver sección 16).

12. Internacionalización y accesibilidad

Idiomas

- `flutter gen-l10n` con `lib/l10n/arb/app_es.arb` como plantilla y `app_en.arb`; el código generado va en `lib/l10n/gen/` y está versionado.
- En widgets se usa `context.l10n`. Fuera del árbol (notificaciones) se usa `platformL10n()`.
- La app sigue el idioma del sistema. En Android 13+ e iOS se puede elegir un idioma solo para la app desde la configuración del sistema: `android/app/src/main/res/xml/locales_config.xml` y `CFBundleLocalizations` en `Info.plist`.
- **Para agregar un texto:** sumarlo a los dos ARB (con su `@descripción` en el de español) y correr `fvm flutter gen-l10n`.

Accesibilidad

- **Filas:** cada una se anuncia como botón con nombre y país ("Abyssinian. País: Ethiopia.") y la pista "ver ficha".
- **Avisos:** los estados de carga, error y vacío, el banner de red y el dato curioso son *live regions*.
- **Buscador e interruptores:** el buscador tiene etiqueta propia; los interruptores de Ajustes se leen como un solo control.
- **Efectos visuales:** el texto con glitch y la máquina de escribir exponen el texto completo una sola vez.
- **Pruebas:** las guías de Flutter (`androidTapTargetGuideline`, `iOSTapTargetGuideline`, `labeledTapTargetGuideline`, `textContrastGuideline`) corren en modo claro y oscuro.
- **Revisión manual:** se revisó el árbol de accesibilidad de Android con TalkBack activado en el emulador. Esa revisión encontró que las filas no se podían activar con TalkBack; está corregido y cubierto por una prueba.

13. Rendimiento

- **Lista:** `SliverPrototypeExtentList` con filas de alto fijo, medido con un prototipo, así la lista no mide fila por fila. La siguiente página se pide cuando faltan 700 px para el final.
- **Impeller y la GPU:** Impeller no tiene raster cache, así que el fondo y los avatares se volvían a pintar en la GPU en cada frame. Ahora se rasterizan una sola vez a textura (`Picture.toImageSync`); los avatares de la lista pasan por un LRU de 120 texturas y en la ficha siguen siendo vectoriales para que el Hero escale nítido.
- **Resultado:** en la traza de 32 flings, los frames que pasan el presupuesto de 16.7 ms bajaron de 16 a 0. El raster promedio del `PerformanceOverlay` bajó de 16.2 a 1.5 ms.
- **Tamaño:** APK arm64 de 20.4 MB. Para reducirlo se cambió la base de zonas horarias por la de 10 años (de 451 a 80 KB) y los efectos de WAV a MP3 (de 323 a 37 KB).

La evidencia completa (tablas, gráficas, capturas del overlay y `analyze-size`) está en el [README](#) y en `docs/performance/`. Para repetir la traza (con un dispositivo o emulador conectado):

```
fvm flutter drive --profile --no-dds --flavor prod \  
  --driver=test_driver/perf_driver.dart \  
  --target=integration_test/scroll_performance_test.dart
```

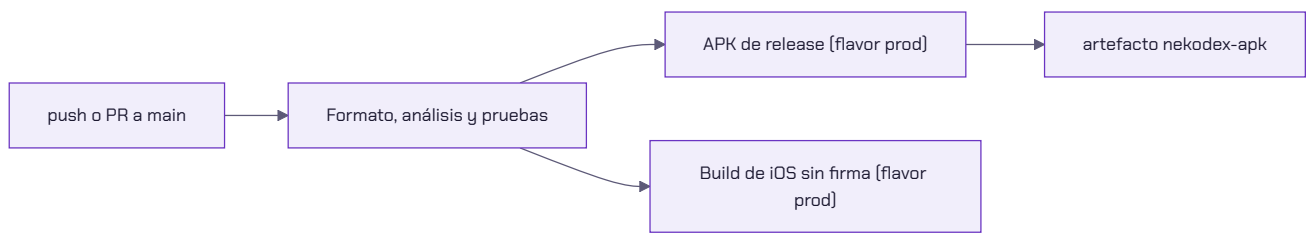
14. Pruebas

87 pruebas unitarias y de widgets (`fvm flutter test`), con un 66 % de cobertura del código propio (sin contar el generado).

Área	Qué cubren
Red	Reintentos, crecimiento exponencial, <code>Retry-After</code> , límite de intentos y métodos no idempotentes
Caché	Frescura, vencimiento, sobre con versión de esquema
Repositorio de razas	Stale-while-revalidate con caché fresca, vieja, vencida y sin red; respaldo a caché; <code>findBySlug</code> ; purga
Fuente remota	Parseo y peticiones compartidas
<code>BreedsBloc</code>	Caché y luego red, errores bloqueantes vs. avisos, paginación sin duplicados, página lenta tras un refresh, búsqueda con debounce, reintento al volver la red
Dato curioso	Repositorio (sin repetir el último) y bloc
Widgets	Estados de la pantalla, banner de reconexión, accesibilidad de las filas
Rutas	Normalización de <code>/breed/:name</code> , pila de un deep link y 404
Ajustes y raza del día	Persistencia, permisos y programación
Ambientes	Flavor → <code>AppEnvironment</code> , la cinta, y que cada ambiente exista en Android e iOS

- **Ayudas** (`test/helpers/`): `FakeClock`, `FakeNetworkInfo` y DTOs de ejemplo en `fakes.dart`, `pump_app.dart` monta widgets con tema, traducciones y "Reducir movimiento" activo para que las animaciones no alarguen las pruebas.
- **Integración:** `integration_test/scroll_performance_test.dart` carga el directorio completo y traza el scroll en modo profile.

15. Integración continua y entrega



Job	Runner	Pasos
Formato, análisis y pruebas	ubuntu-latest	<code>pub get</code> , regenera código y traducciones y verifica que no haya cambios, <code>dart format --set-exit-if-changed</code> , <code>flutter analyze</code> , <code>flutter test --coverage</code>
APK de release	ubuntu-latest + JDK 17	<code>flutter build apk --release --flavor prod</code> y sube el APK como artefacto
Build de iOS	macos-26 [Xcode 26]	<code>flutter build ios --release --no-codesign --flavor prod</code>

La versión de Flutter de la CI es la misma que fija `.fvmrc` .

Firma y publicación

- **Android:** si existe `android/key.properties` (no versionado), el release se firma con esa clave; si no, con la de debug. El APK de Releases está firmado con la clave de debug del equipo de desarrollo. Al pasar a una clave propia hay que actualizar la huella en `assetlinks.json` , en el repositorio y en el sitio publicado.
- **iOS:** equipo `Y6QAJXAGX6` con firma automática. La primera vez que se compila DEV o QA para un dispositivo, Xcode registra sus ids (`.dev` y `.qa`) en ese equipo.
- **Publicar una versión:**
 1. Subir `version` en `pubspec.yaml` y `AppConfig.version` .
 2. `fvm flutter build apk --release --flavor prod` .
 3. Crear el release en GitHub con el APK adjunto.

16. Mantenimiento

Tarea	Cómo
Cambiar modelos (freezed / JSON)	Editar y correr <code>fvm dart run build_runner build --delete-conflicting-outputs</code> ; la CI falla si el generado no está al día
Cambiar textos	ARB + <code>fvm flutter gen-l10n</code>
Cambiar la forma de lo guardado en caché	Subir <code>cacheSchemaVersion</code> en <code>lib/app/di/injection.dart</code>
Animaciones Lottie y sonidos	<code>tool/assets/lottie_gen.py</code> (solo Python 3) y <code>tool/assets/sfx_gen.py</code> (Python 3 y ffmpeg)
Logo, ícono y splash	<code>tool/branding/render_brand.py</code> genera los PNG con Chrome sin interfaz. Después <code>flutter_launcher_icons</code> y <code>flutter_native_splash</code> : revisar el diff , porque reescriben <code>project.pbxproj</code> e <code>Info.plist</code>
Gráficas de la auditoría	<code>tool/perf/frames_chart.py</code> a partir de los JSON de <code>docs/performance/</code>
Manuales en PDF	<code>python3 tool/docs/build_manuals.py</code> : los exporta con Chrome sin interfaz (necesita red para marked y mermaid)
Actualizar Flutter	<code>fvm use <versión></code> (actualiza <code>.fvmrc</code> y la ruta de VS Code) y <code>FLUTTER_VERSION</code> en <code>.github/workflows/ci.yml</code>

Tarea	Cómo
Cambiar el dominio de los deep links	Ver deeplinks/README.md

17. Problemas conocidos y soluciones

Síntoma	Causa	Solución
La app se cierra al arrancar en el simulador con <code>Failed to load dynamic library 'objective_c.framework'</code>	Flutter 3.41 guarda la librería nativa de <code>path_provider</code> en una sola carpeta (<code>build/native_assets/ios</code>) para iPhone y simulador, y reutiliza la del último build	Tarea <code>Clean</code> , o borrar <code>build/native_assets</code> y <code>.dart_tool/flutter_build</code>
El build de iOS falla con Xcode 16	<code>connectivity_plus</code> 7 usa APIs del SDK de iOS 26	Xcode 26 (en la CI, <code>macos-26</code>)
Profile o release no arrancan en el simulador de iOS	Flutter solo los admite en dispositivos físicos	Usar un iPhone o el modo debug
<code>pod install</code> cambia <code>Podfile.lock</code> sin agregar dependencias	Cualquier cambio en el <code>Podfile</code> , incluso un comentario, cambia su checksum	Correr <code>pod install</code> y versionar el <code>Podfile.lock</code>
El link <code>https</code> abre el navegador en un APK de la CI	Ese APK se firma con la clave del runner, que no está en <code>assetlinks.json</code>	Usar el APK de Releases o agregar la huella

18. Decisiones de diseño y límites

- **Hive CE en lugar de sqflite:** es Dart puro, lee de memoria tras abrir la caja y la caché es un JSON por página; no hacía falta SQL.
- **Se guardan todas las páginas visitadas,** no solo la primera: así el directorio completo funciona sin conexión.
- **La búsqueda es local,** como pide la prueba: filtra lo cargado y ofrece cargar más.
- **Flavors nativos y no `--dart-define-from-file`:** el flavor cambia id, nombre y esquema en cada plataforma. En Flutter 3.41 los valores de `--dart-define-from-file` solo llegan a Gradle y a Xcode codificados en `DART_DEFINES`, así que lograr lo mismo exigía decodificarlos en los scripts de build.
- **Datos curiosos en inglés:** así los publica la API; la ficha lo aclara.
- **Métricas de rendimiento de emulador:** la GPU emulada es más lenta que la de un teléfono; conviene repetir la traza en hardware antes de dar cifras de producción.
- **Pendiente:** una pasada con VoiceOver y la prueba de notificaciones en un iPhone físico.